Individual flutter lab 5

Post manager with SqLite

Q1. In this flutter project, the following dependencies are used.

* SqLite : which is used to manage a local SQLite database. Also it allows storing data directly on the device without internet.

* path : Used to define the correct file path where the database is stored.

b. The SqLite is necessary because : it allow offline data storage, data remain saved even after the app is closed.

It is lightweight and built into mobile device. Also it is suitable for apps like:
- Notes app
- offline post managers
- Task managers.

In my project, the SqLite is used to store posts (title, content, favorite status) locally.

Q2. The handling errors is important to prevent crashes

1. Database not initialized: App try to use database before it created.
Its solution is to use a singleton pattern

```
If (_database != null) return _database!;
```

Which ensure that database is created only once and reused.

2. Insert / Update / delete error : operations may fail (invalid data, connection issue)

The solution: is use try - catch.

```
try {
  await db.insert('posts', post);
} catch (e) {
  print ("insert error: $e");
```

* prevent the app from crashing.

3. Invalid or Corrupted data : the data might be wrong or incorrect type

The solution is Validating before Saving

```
if ( title.isEmpty || content.isEmpty)
    return;
```

to ensure only valid data is stored.

Q3. 1. Database Vs table

Database	table
* A container that hold all data	* A structure inside the database.
* Ex: post.db	* Store records in rows and columns
	* Ex: table: post - id - title - content - favorite.

2. Crude operations. it stands for Create, read, Update and delete.

* Create it is like add new data

```
db.insert('posts', data);
```

* Read, get data

```
db.query('posts');
```

* Update, Modify data

```
db.Update('posts', data);
```

* Delete : Remove data

```
db.delete('posts');
```

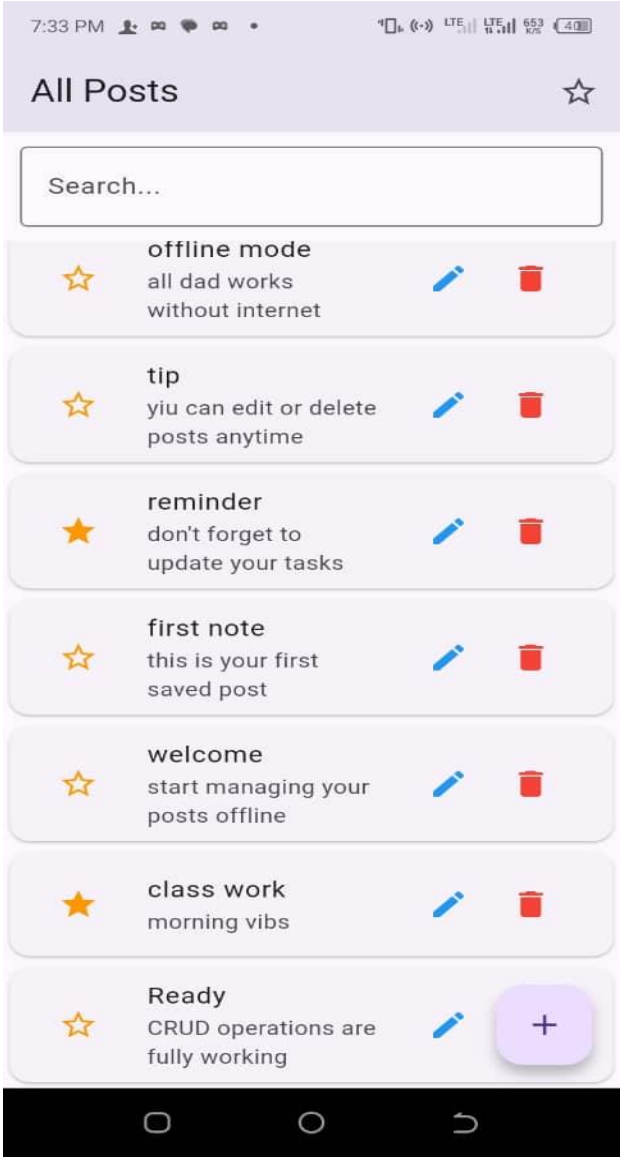
3. Asynchronous Interaction in flutter.

flutter Uses async / await becoose database operation take time.

```
flutter <list <Map < string, dynamic>>>
  get posts() async {
    final db = await database;
    return await db.query ('posts');
  }
```

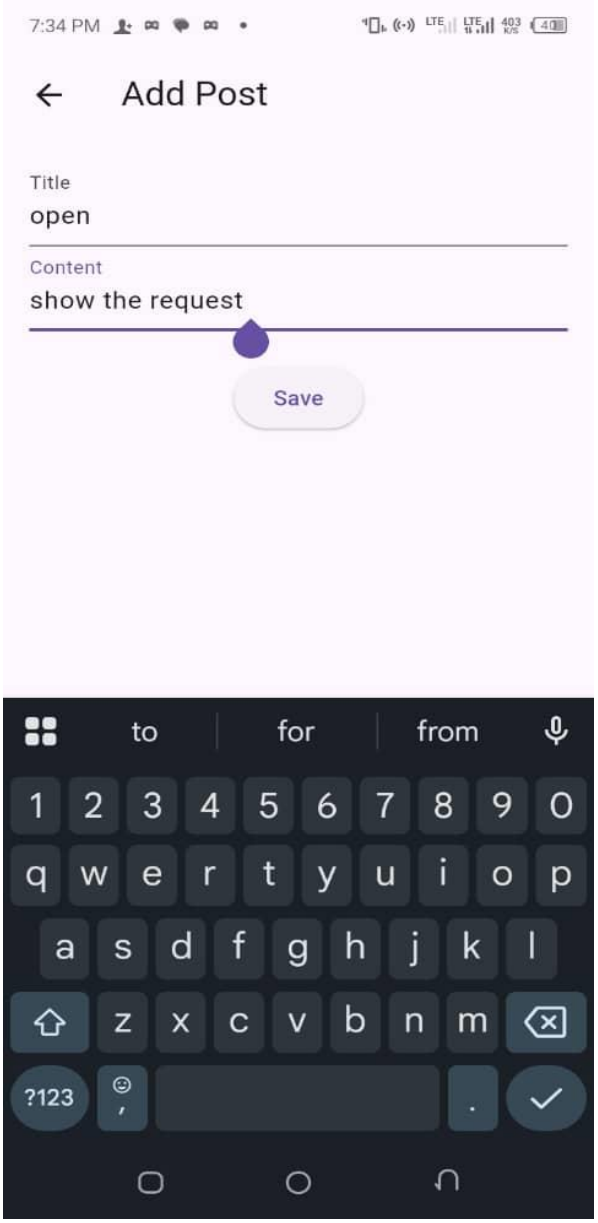
The Async it - prevent app freezing
- keep UI smoth
- Allow background data processing.

1. HOME SCREEN (All Posts)

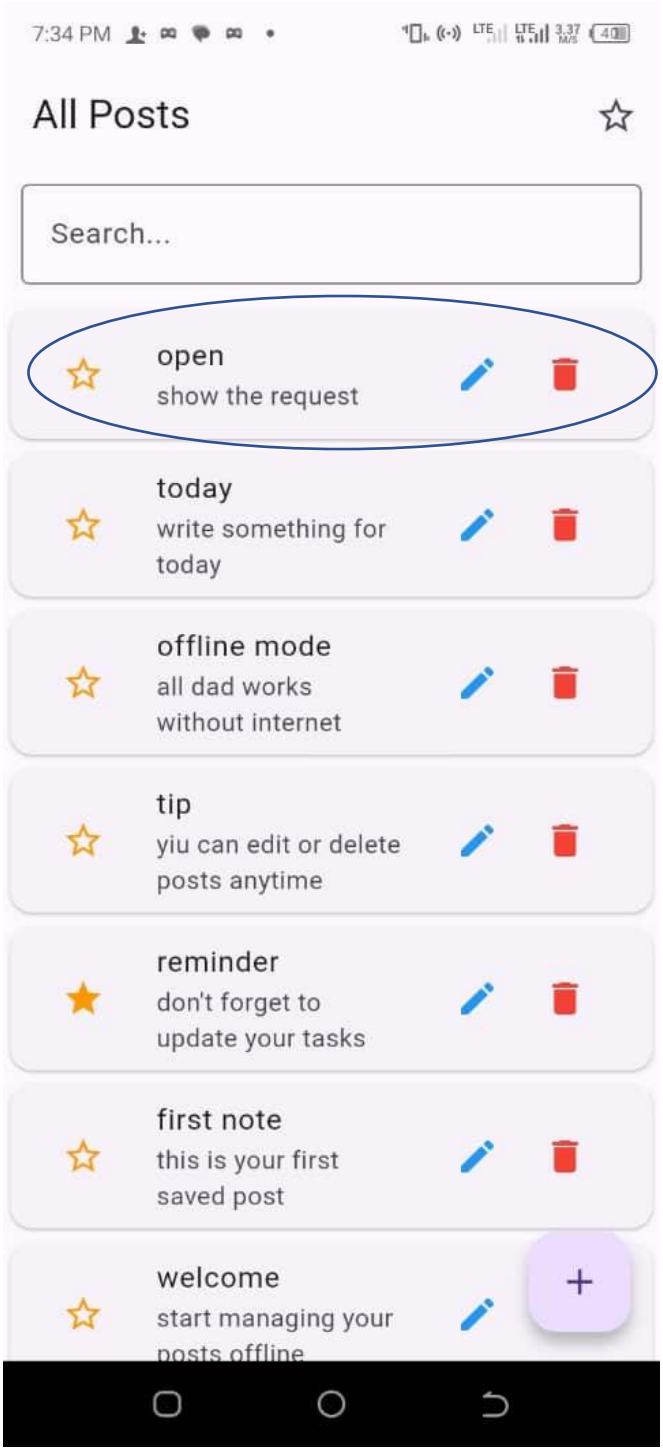


This screen displays all stored posts from the local SQLite database. Each post contains a title, content, and a favorite indicator. Data is loaded asynchronously when the app starts.

2. ADD POST SCREEN

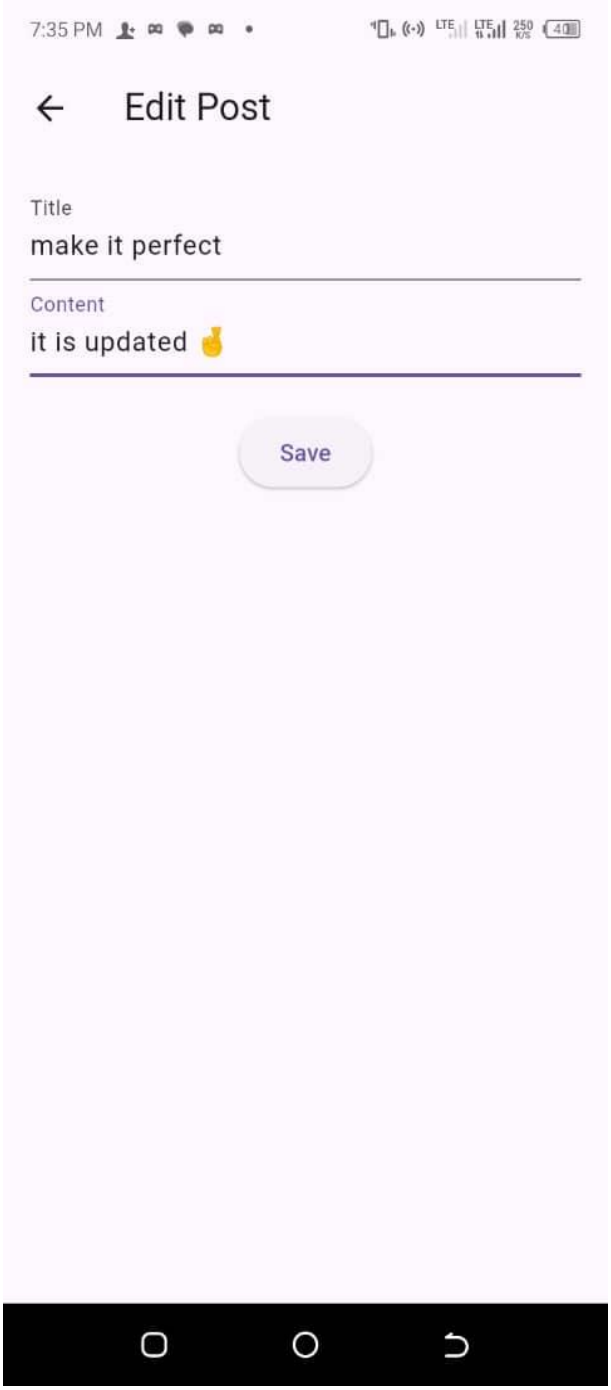


This screen allows the user to create a new post. The data entered is saved into the SQLite database using an insert operation.

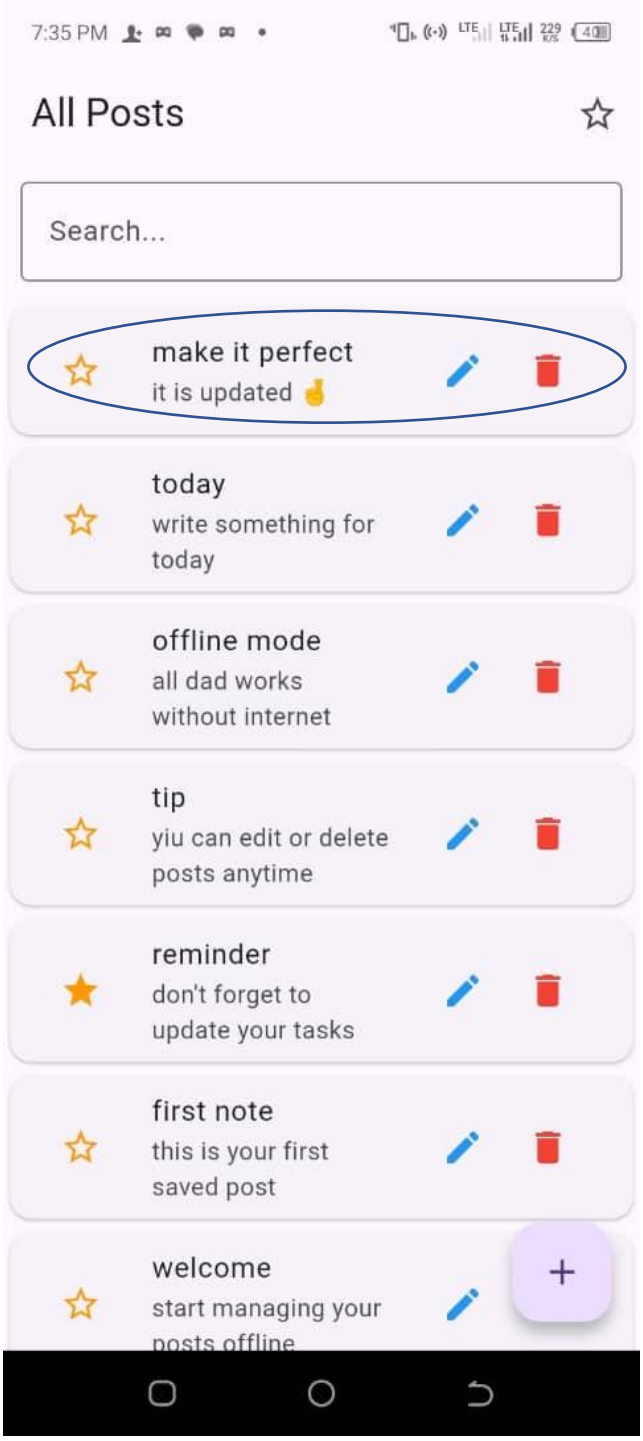


I am adding a new post titled **"Open"**, as shown in the screenshot above

3. EDIT POST

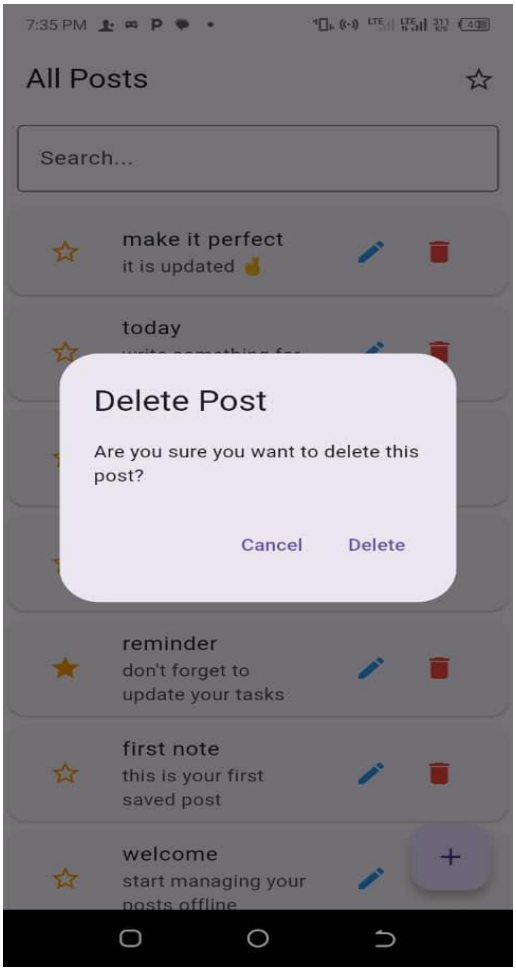


This screen allows editing an existing post. The previous data is loaded and updated using the update operation in the database.



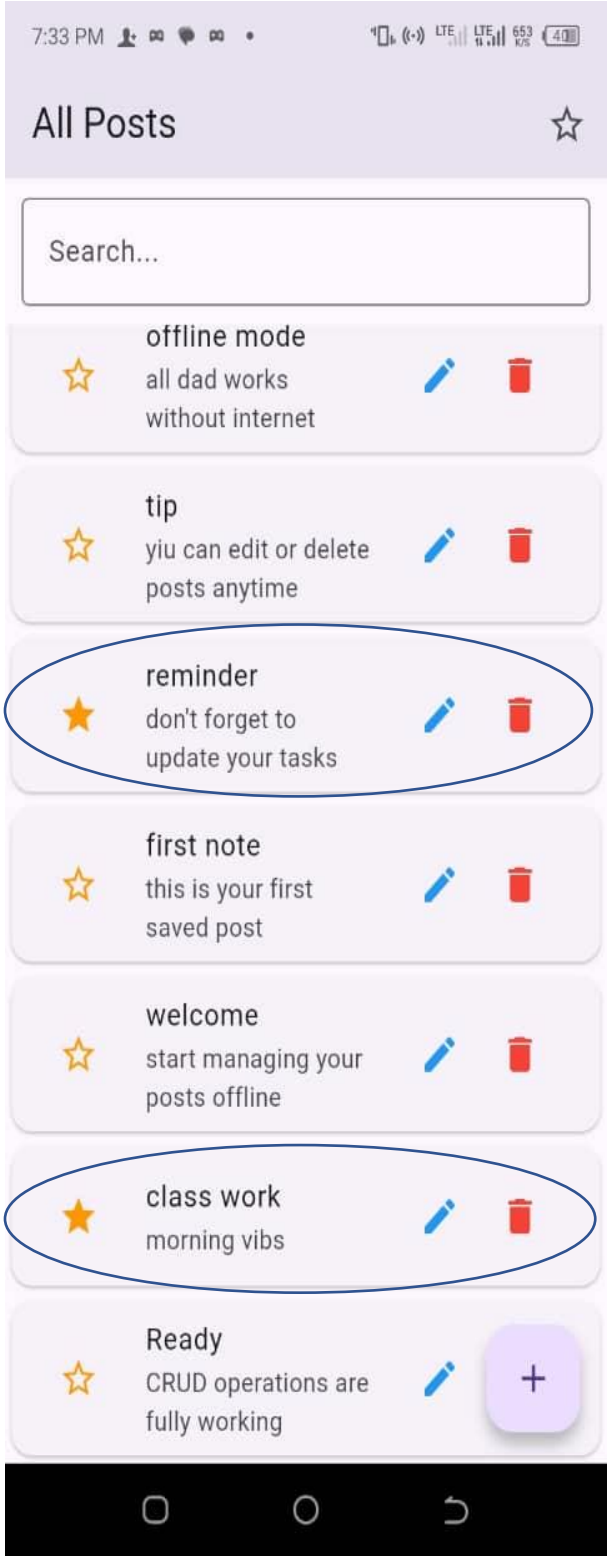
Also, the open post is edited to make it perfect post

4. DELETE CONFIRMATION



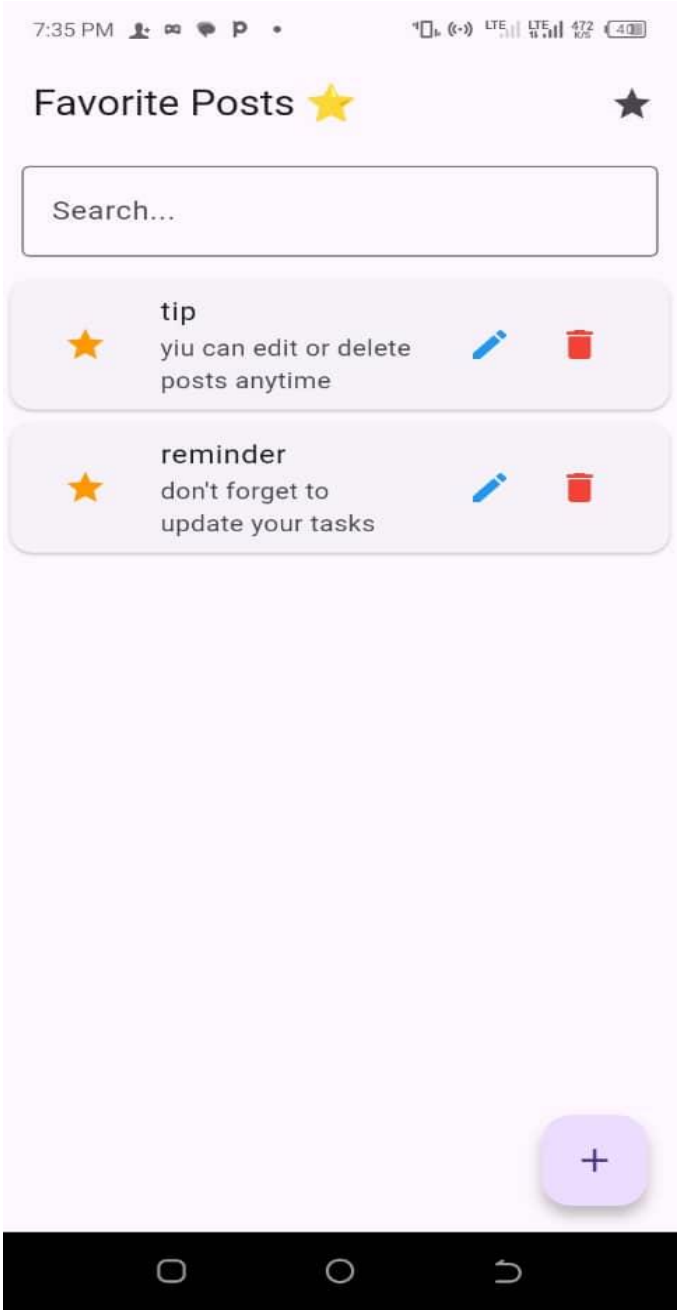
This dialog ensures that the user confirms before deleting a post. It prevents accidental data loss and improves user experience.

5. FAVORITE FEATURE



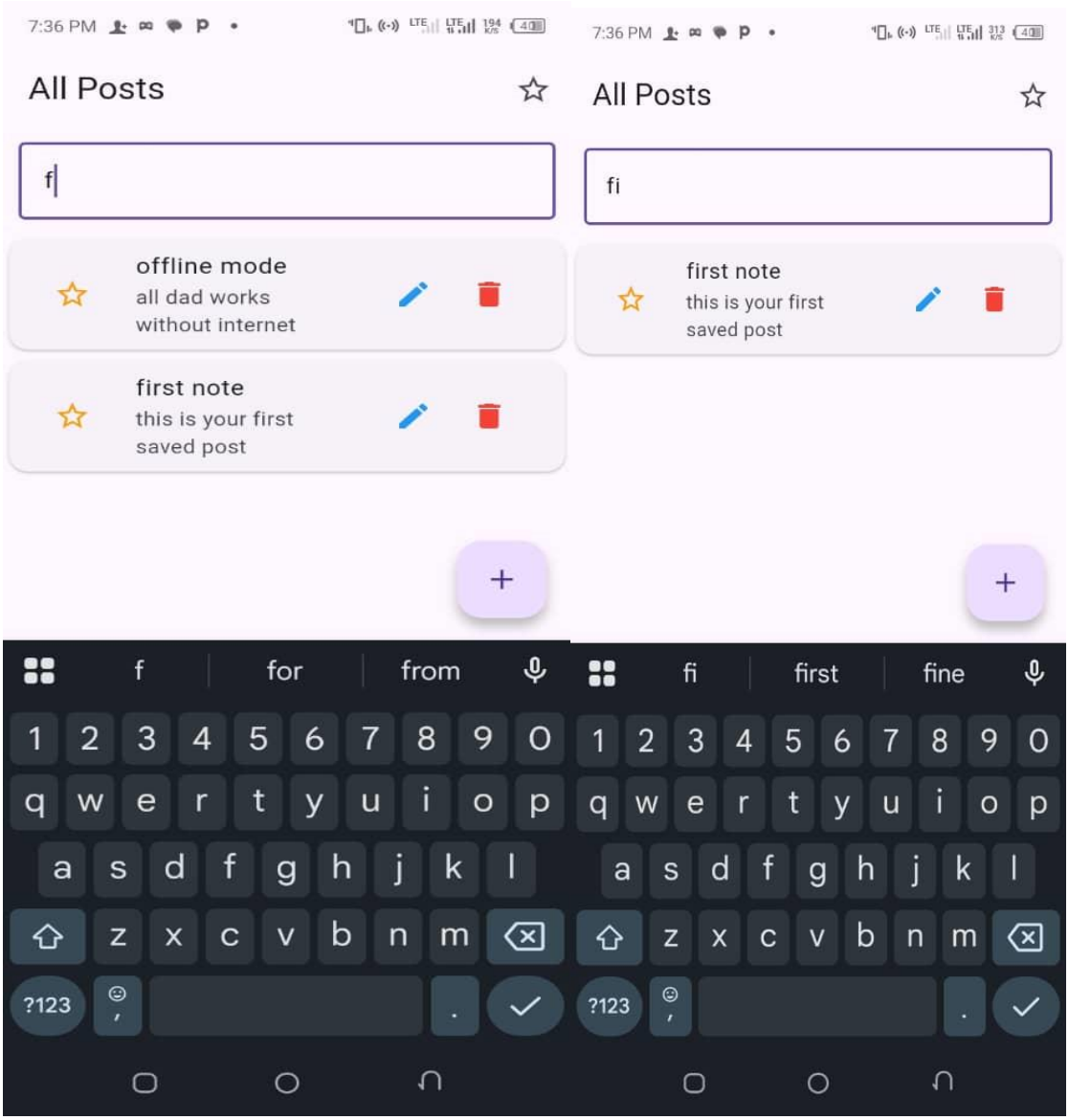
This feature allows users to mark important posts as favorites. The favorite status is stored in the database and can be toggled dynamically. There are two post that marked as favorite post

6. FAVORITE FILTER (TOP BUTTON)



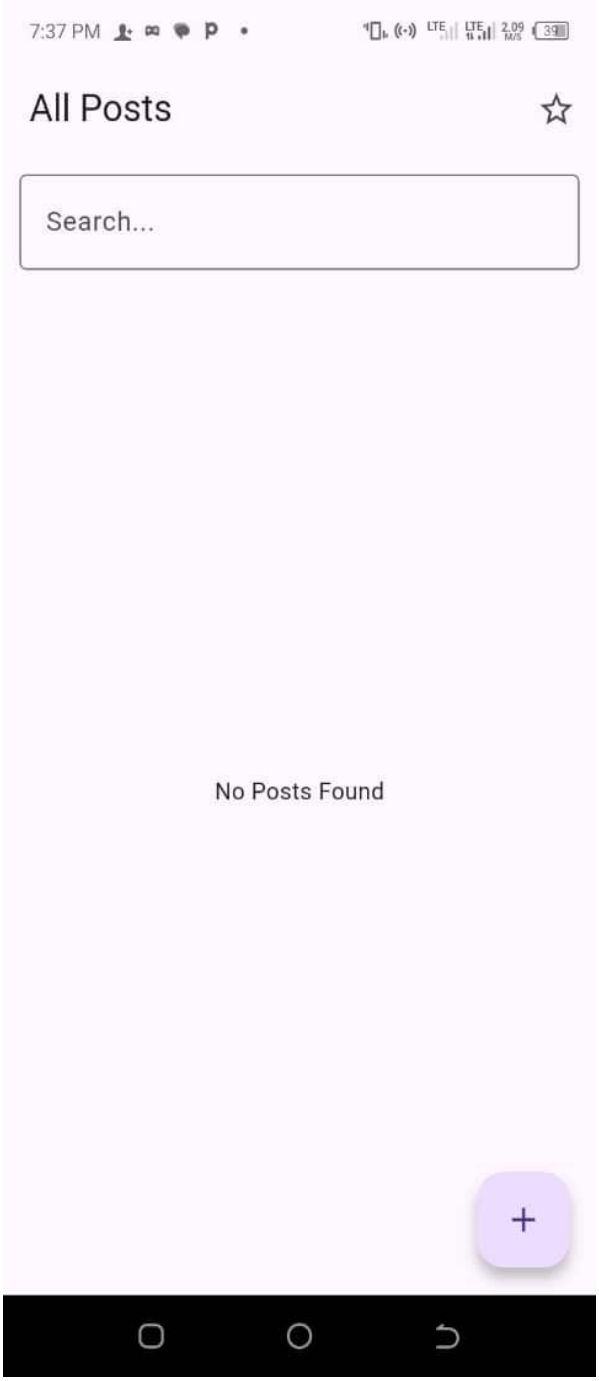
This feature filters and displays only favorite posts. It improves usability by allowing users to quickly access important data. My favorite post is selected all of it.

7. SEARCH FUNCTION



This feature allows users to search posts by title. It filters results dynamically and improves navigation within stored data.

8. EMPTY STATE



This screen is displayed when no data matches the search or filter. It provides clear feedback to the user.

CODE

```dart
// database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('posts.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  Future _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE posts(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT,
        content TEXT,
        isFavorite INTEGER
      )
    ''');

    // Default posts
    await db.insert('posts', {
      'title': 'Welcome',
      'content': 'Start managing your posts offline.',
      'isFavorite': 0
    });

    await db.insert('posts', {
      'title': 'Study Tip',
      'content': 'Review your notes daily.',
      'isFavorite': 1
    });

    await db.insert('posts', {
      'title': 'Reminder',
      'content': 'Submit your assignment on time.',
      'isFavorite': 0
    });
  }

  Future<List<Map<String, dynamic>>> getPosts() async {
    final db = await instance.database;
    return await db.query('posts', orderBy: 'id DESC');
  }

  Future<int> insertPost(Map<String, dynamic> post) async {
    final db = await instance.database;
    return await db.insert('posts', post);
  }

  Future<int> updatePost(int id, Map<String, dynamic> post) async {
    final db = await instance.database;
    return await db.update(
      'posts',
      post,
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  Future<int> deletePost(int id) async {
    final db = await instance.database;
    return await db.delete(
      'posts',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
```

This code shows the implementation of SQLite database operations including table creation and CRUD functionality.